Decathlon VOC Analyzer：面向商品图文证据与用户评论对齐的证据驱动多模态 VOC 分析系统

Severin Ye
School of Computer Science and Engineering
Kyungpook National University
Daegu, Republic of Korea
6severin9@gmail.com

Seowan Jung
School of Computer Science and Engineering
Kyungpook National University
Daegu, Republic of Korea
swan041014@gmail.com

HyunJun Jung
School of Computer Science and Engineering
Kyungpook National University
Daegu, Republic of Korea
beanbeansummon@naver.com

Hye Jeon
School of Computer Science and Engineering
Kyungpook National University
Daegu, Republic of Korea
prajna0426@naver.com

Dokeun Lee
Department of Physics
Kyungpook National University
Daegu, Republic of Korea
nsa08008@naver.com

Abstract

电商商品的用户之声（Voice of Customer, VOC）分析需要同时理解用户评论中的主观体验和商品页面中的客观证据。仅依赖评论文本的情感或方面分析，难以判断用户观点是否被商品文案、规格或图像支持；直接把评论和商品图文整体交给大语言模型生成总结，又容易产生缺乏证据链的黑盒结论。针对这一问题，本文提出 Decathlon VOC Analyzer，一个面向商品图文证据与用户评论对齐的证据驱动多模态 VOC 分析系统。

系统将原始商品目录标准化为包含商品文本、商品图片、图片局部区域和评论记录的证据包；在评论侧执行低信息过滤、星级抽样、方面级结构化抽取和去重；再将每个评论方面规划为具有明确检索意图和模态路线的问题；随后在商品文本和商品图像索引中执行两阶段召回与重排；最终生成包含优势、缺点、争议、证据缺口、改进建议和 claim 级证据归因的结构化报告。系统同时提供 FastAPI 接口、LangGraph 单产品工作流、运行策略、检索缓存、feedback/replay 侧边车、HTML 导出和 manifest 评估模块。

本文的主要贡献包括：第一，提出一种 “评论方面建模—问题规划—多模态证据检索—证据约束生成” 的商品 VOC 分析框架；第二，实现一套以结构化 schema 和 artifact 为中心的可复现系统，使评论、问题、证据、报告和评估指标均可追踪；第三，给出面向系统验证的实验协议，当前代码基线下 166 项自动化测试全部通过，并支持 Recall@K、MRR、NDCG、claim support rate 和 citation precision 等指标。本文定位为系统方法论文，重点论证系统架构、实现机制和可复现实验接口，而不声称已经完成覆盖多品类的大规模人工标注 benchmark。

关键词：多模态 RAG；用户之声分析；方面级评论建模；证据约束生成；商品评论分析；证据归因

1 引言

用户评论分析是电商智能运营中的基础任务。传统方法通常围绕情感分类、主题聚类或方面级情感分析展开，能够概括用户对商品属性的正负态度，却较少回答一个更关键的问题：这些评论观点是否能够被商品本体证据解释、支持或反驳。对于商品运营和产品改进而言，单纯知道“用户抱怨容量小”并不足够，分析者还需要知道商品页面是否明确描述了尺寸、图片是否展示了空间结构、该问题是商品缺陷还是用户预期偏差。

大语言模型为商品评论总结提供了新的可能，但一步式总结同样存在明显风险。如果系统直接将商品文案、图片描述和评论整体输入模型，模型可能生成流畅但难以审查的结论。使用者很难判断某条建议来自哪些评论、哪些商品文本或哪张图片，也难以定位错误发生在评论理解、检索、证据解释还是最终生成阶段。因此，商品 VOC 分析不应被简化为黑盒摘要任务，而应被建模为一条从评论需求到商品证据、再到结构化报告的可追踪链路。

本文研究的问题是：如何在电商商品场景中，将用户评论中的主观体验显式对齐到商品文案和商品图像中的可核验证据，并生成可审查、可复现、可评估的 VOC 洞察。为此，本文提出 Decathlon VOC Analyzer。系统首先以商品为中心构建标准化证据包，保留商品文本、商品图片、图片局部区域和评论记录；随后将评论抽取为方面级对象；再通过问题规划层把每个方面转化为文本、图像或图文联合检索任务；最后通过多模态召回、重排、证据聚合和 claim 级归因生成结构化报告。

本文的贡献可以概括为三点。

第一，本文提出一种面向商品 VOC 的证据驱动多模态 RAG 框架。不同于直接用评论原句检索商品证据，本文在评论方面和商品证据之间加入问题规划层，使主观表达被转化为可执行的证据查询。

第二，本文实现一套 artifact-first 的系统原型。系统中的商品证据、评论方面、检索问题、召回记录、报告洞察、claim attribution、feedback、replay 和 manifest 均以结构化 schema 表示并可落盘复用。

第三，本文给出可复现实验接口和评估设计。当前实现包含 API、批处理脚本、HTML 导出、manifest 评估和自动化测试；评估模块支持检索指标与生成证据支撑指标，从而同时衡量“是否找到了证据”和“生成结论是否被证据支持”。

本文其余部分组织如下：第 2 节给出背景和问题定义；第 3 节讨论相关工作；第 4 节详细介绍系统方法；第 5 节说明实验设置与实现环境；第 6 节报告系统验证结果；第 7 节讨论方法意义；第 8 节总结全文；第 9 节说明局限性。

2 背景与问题定义

2.1 商品 VOC 分析的证据需求

面向商品的 VOC 分析与通用评论摘要不同。商品分析通常需要输出稳定的洞察对象，包括商品优势、主要缺点、争议点、适用场景、证据缺口和改进建议。每个洞察都应尽可能回指到两类证据：一类是用户评论中的原始表达，另一类是商品页面中的文本、图片或局部视觉结构。

如果系统只处理评论，它只能回答“用户怎么说”；如果系统只处理商品页面，它又无法知道“用户真正关心什么”。因此，商品 VOC 分析的关键在于把评论侧的主观需求和商品侧的客观证据连接起来。本文将这一连接建模为方面驱动的证据检索问题，而不是直接生成总结。

2.2 输入与输出

给定一个商品数据包 P，其包含商品文本 P_{text}、商品图像 P_{image} 和评论集合 $C_{reviews}$。系统目标是输出结构化报告 R，其中包括报告主张、支持证据、证据缺口和改进建议。该过程可表示为：

$$R = F(P_{text}, P_{image}, C_{reviews})$$

但本文中的 F 不是单步生成函数，而是一条分阶段链路：

$$C_{reviews} \rightarrow A_{aspects} \rightarrow I_{intents} \rightarrow Q_{questions} \rightarrow E_{retrieval} \rightarrow G_{aggregates} \rightarrow R_{attributed}$$

其中，$A_{aspects}$ 表示评论方面对象，$I_{intents}$ 表示问题意图，$Q_{questions}$ 表示可执行检索问题，$E_{retrieval}$ 表示商品图文证据，$G_{aggregates}$ 表示方面聚合统计，$R_{attributed}$ 表示带证据归因的最终报告。

2.3 设计约束

本文系统建立在五项约束之上。

第一，商品证据必须对象化。商品标题、描述、类目、图片、图片区域和评论不能只作为拼接文本输入模型，而应拥有稳定 ID 和来源字段。

第二，评论侧必须先结构化。系统需要把评论拆解为 aspect、sentiment、opinion、evidence span、usage scene 和 confidence，才能支持后续统计与检索。

第三，检索查询必须问题化。评论原句往往口语化且噪声高，直接检索容易产生宽泛候选；问题规划能够将其转化为可核验的证据需求。

第四，文本证据与图像证据应分 route 管理。商品文案和商品图片支持不同类型的判断，系统需要保留 route 信息用于召回、重排和归因。

第五，最终报告必须可追踪。报告中的主张应能映射到评论 ID、文本块 ID、图片 ID 或区域 ID；证据不足时应输出 evidence gap，而不是强行确认结论。

2.4 与通用 RAG 问答的差异

通用 RAG 系统通常以用户提出的问题为查询入口，目标是生成单个答案。本文任务的查询入口来自评论方面，目标不是单一回答，而是商品 VOC 画像。系统需要同时处理多个方面、多个问题、多个证据路线和多个报告字段，并保留完整过程 trace。因此，本文更接近一个面向商品分析的证据编排系统，而不是一个普通问答系统。

3 相关工作

3.1 检索增强生成与证据约束

检索增强生成（Retrieval-Augmented Generation, RAG）将外部检索与参数化生成分离，使生成模型能够基于显式证据回答知识密集问题 [1]。后续研究进一步表明，检索不仅可以作为生成前的上下文补充，也可以用于生成后的校正与修订，例如 RARR 通过外部搜索和修订降低模型输出中的不实陈述 [2]。这些工作为本文提供了基本方法论：商品 VOC 报告不应完全依赖模型内部知识，而应围绕可检索商品证据生成。

本文与通用 RAG 的不同在于，检索查询不是用户即时问题，而是由评论方面规划得到的证据问题；检索对象也不是百科段落，而是商品文本、商品图片和图片区域。最终目标不是单条答案，而是带 claim 级证据归因的结构化商品分析报告。

3.2 多模态检索与视觉证据

CLIP 证明了图像和文本可以通过自然语言监督映射到共享语义空间，从而支持跨模态检索 [3]。这一思想为商品图片作为一等证据对象提供了基础。多模态 RAG 的工程实践也强调，真实业务数据通常由文本、图像、表格和结构化字段共同构成，系统需要同时处理模态表示、跨模态检索和上下文融合 [4, 5]。

视觉文档检索和原始模态召回研究进一步指出，图像不应过早被压缩为纯文本。ColPali 等方法将文档页面作为视觉对象进行检索，并通过多向量表示和 late interaction 提升细粒度匹配能力 [6]。本文没有直接实现 ColPali 式多向量检索，但吸收了保留原始视觉证据和向区域级证据扩展的思想，在商品图片层面同时建模整图和默认局部区域。

3.3 评论分析与方面建模

方面级情感分析通常将评论拆解为属性、观点和情感极性，适合发现用户集中关注的商品特征 [7]。对于商品 VOC 分析而言，方面建模仍是必要入口，因为评论经常包含多种体验、场景和情绪。如果系统不先进行方面级建模，后续检索和报告生成就只能依赖冗长评论原文。

本文与传统方面级情感分析的差异在于，方面不是最终结果，而是证据检索的起点。每个方面都会被转化为一个或多个带检索路线的问题，并进一步连接商品文本和商品图像证据。

3.4 两阶段检索、向量库与重排

实际检索系统常采用粗召回和精排分离的多阶段架构。粗召回负责覆盖候选，精排负责在小候选集上使用更高成本模型提高排序质量。Qdrant 的多阶段检索实践和 multi-vector search 课程均强调，在复杂检索任务中，向量库应作为候选管理基础设施，而不应替代后续 reranker 和业务判断 [8, 9]。

本文采用这一思想，将 embedding、索引后端、候选池构建和 reranker 解耦。文本候选和图像候选分别处理，最终在报告层统一归因。

3.5 结构化输出与工作流编排

LLM 在结构化输出任务中可能出现字段缺失、类型错误和格式漂移，因此需要 schema 约束、JSON mode 或语法约束等机制提高稳定性 [10, 11]。ReAct 等工作则强调推理和行动之间的显式交替，说明复杂任务更适合拆为多步过程而非单轮提示 [12]。

本文系统沿用这一方向：评论抽取、问题生成、报告生成均以结构化 schema 为边界；端到端流程则由 LangGraph 状态机组织为概览、标准化、索引和分析等节点。

3.6 本文定位

本文不提出新的通用 embedding 模型或 reranker 算法，而是提出一个面向商品 VOC 的系统化整合框架。其核心创新在于把评论方面、问题规划、多模态证据检索、结构化报告和 claim 级归因组织为一个可复现闭环，使商品分析从黑盒总结转向可审查的证据工作流。

4 方法

4.1 总体架构

Decathlon VOC Analyzer 采用分阶段的证据驱动架构。系统主链路包括：数据集与商品证据标准化、评论建模与抽样、问题规划、多模态索引与召回、重排与缓存、报告生成与证据归因、工作流编排和评估。各阶段之间通过 Pydantic schema 传递对象，而不是传递无结构文本。

单商品工作流由 LangGraph 编排为四个节点：`overview`、`normalize`、`index` 和 `analyze`。`overview` 统计数据集规模与缺失文件；`normalize` 将目标商品转化为标准化证据包；`index` 为商品文本、图片和区域建立索引；`analyze` 执行评论抽取、问题生成、检索、重排、聚合、报告生成和产物落盘。该流程保证系统既可以通过 API 交互调用，也可以通过批处理脚本复现实验。

4.2 商品证据标准化

数据标准化模块以 `ProductEvidencePackage` 为核心对象。系统从每个商品目录读取 `product.json`、`reviews.json` 和 `images/`，将商品标题、描述和类目转化为 `TextEvidence`，将图片转化为 `ImageEvidence`，将评论转化为 `ReviewRecord`。每个证据对象都具有稳定 ID、来源字段和语言提示。

为了为局部视觉证据预留空间，系统对有效图片生成五类默认区域：`center_focus`、`upper_focus`、`lower_focus`、`left_focus` 和 `right_focus`。这些区域由相对比例转换为像素框，并被建模为 `ImageRegion`。当前区域并不等价于语义检测结果，但它使系统能够在没有人工标注框的情况下初步支持区域裁剪、局部 embedding 和图像重排。

标准化过程还会输出数据质量信息，包括商品数量、评论数量、图片数量、缺失 `product.json` 或 `reviews.json` 的商品，以及空评论数量。标准化产物写入 `02_outputs/1_normalized/`，后续所有模块优先复用该产物。

4.3 评论建模与抽样

评论建模模块将原始评论转化为 `ReviewAspect`。每个方面对象包含 `aspect`、`sentiment`、`opinion`、`evidence_span`、`usage_scene`、`confidence` 和 `extraction_mode`。系统首先清洗评论文本，过滤空评论、过短评论和低信息短评，例如 `ok`、`good`、`great` 等。

当设置 `max_reviews` 时，系统启用星级抽样。默认 profile 为 `problem_first`，其目标比例为 1 星 30%、2 星 25%、3 星 20%、4 星 15%、5 星 10%。如果某个星级样本不足，系统按配置的 fallback 顺序补位，并将目标数量、可用数量、实际选择数量、短缺数量和补位数量写入 `ReviewSamplingPlan`。

方面抽取支持 LLM 和 heuristic 两条路径。LLM 路径使用 Qwen 网关和结构化输出 schema；heuristic 路径使用关键词、评分和负向提示词识别容量、便携性、可用性、耐用性、价值价格、材料硬度等方面。抽取后，系统按评论 ID、方面、情感和证据片段去重，并将结果写入 `02_outputs/2_aspects/`。

4.4 问题规划

问题规划模块是本文方法的关键桥接层。系统不直接用评论方面或评论原句检索商品证据，而是先为每个 `ReviewAspect` 规划若干 `QuestionIntent`，再生成 `RetrievalQuestion`。默认意图包括：`explicit_support`、`visual_confirmation`、`cross_modal_resolution`、`spec_check` 和 `visual_detail`。

每个问题都包含来源评论、来源方面、自然语言问题、生成理由、期望证据路线和置信度。`expected_evidence_routes` 明确指定检索应使用 `text`、`image` 或二者结合。该字段使后续召回具备 route-aware 能力，也使报告归因能够区分文本支持、图像支持和混合支持。

问题生成同样支持 LLM 和 heuristic 两条路径。LLM 路径将方面、观点、证据片段、使用场景、情感、意图类型、禁止概念和具体性边界输入提示词；heuristic 路径使用固定模板生成可检索问题。问题缓存签名绑定 prompt variant、prompt digest、方面 digest、问题数量、模型和 API 配置，避免提示词变化后误用旧问题。

4.5 多模态索引与召回

索引模块将商品证据转化为 `IndexedEvidence`。文本证据 route 为 `text`，图像整图和区域 route 为 `image`。文本 embedding 可使用 `text-embedding-v4` API 或本地 Qwen3 embedding；图像 embedding 默认使用 `openai/clip-vit-base-patch32`。当外部能力不可用且运行策略允许降级时，系统可退回 hash embedding 或代理文本 embedding。

索引后端通过 `IndexBackend` 抽象统一管理。`LocalIndexBackend` 将索引快照写入本地 JSON 文件，并在搜索时计算 query 向量与 evidence 向量的相似度；`QdrantIndexBackend` 为文本和图像分别创建 collection，并用 `product_id`、`category_slug` 和 `route` 过滤候选。

召回过程针对每个问题执行。系统首先按 route 进行 embedding 粗召回，并使用过采样保留较宽候选集合；随后按语言和 route 构建均衡候选池，避免单一语言或单一路线垄断候选；最后交给 reranker 精排，并通过 route coverage 规则选择最终证据。

4.6 重排、缓存与运行策略

重排模块将文本候选和图像候选分开处理。文本候选支持本地 Qwen3 reranker、DashScope `gte-rerank-v2` API 和启发式排序；图像候选支持本地 Qwen3-VL、多模态 Qwen-VL API、文本 reranker 退路和启发式排序。

在多模态图像重排中，系统会读取商品图片，若候选为区域证据则按 `region_box` 裁剪，然后压缩为 JPEG data URL 输入 Qwen-VL，并要求模型返回严格 JSON 评分。这使图像候选的最终排序能够基于真实视觉内容，而不是只依赖图片路径或代理文本。

系统将 query embedding 和 rerank 结果写入磁盘缓存。缓存签名绑定 route、query、backend、model、base URL 和候选 digest，因此相同配置可以复用结果，不同模型或候选集不会串写。运行策略由 `RuntimeExecutionPolicy` 控制：开发时可允许降级，正式实验时可禁止静默降级，满血模式下还会检查 Qdrant、本地 embedding、本地 reranker 和本地多模态 reranker 等前置条件。

4.7 报告生成、归因与回放

报告生成模块首先对方面进行聚合，形成每个 aspect 的情感分布、代表评论、证据片段和平均置信度。随后系统基于方面聚合和检索结果生成结构化报告，字段包括 strengths、weaknesses、controversies、evidence gaps、applicable scenes 和 suggestions。LLM 可用时进入结构化生成路径；不可用时使用 heuristic 路径构造同 schema 报告。

报告生成后，`ReportRefinementService` 会对洞察和建议去重，依据支持证据类型重新校准 owner 与 evidence level，并修正“声称有图像支持但实际未检索到图像证据”等不一致表述。随后系统构建 `claim_attributions`，将报告主张映射到评论、商品文本、商品图片或区域证据，并标记 supported、partial、unsupported 或 contradicted。

系统还支持 feedback/replay 侧边车。若存在上一轮反馈或回放产物，新分析可以读取这些上下文，将持续存在的问题、已修正问题和回放摘要并入报告与 trace。最终产物包括 analysis JSON、feedback sidecar、replay sidecar、HTML 报告和 run manifest。

4.8 Schema 与可解释性

本文系统的可解释性来自结构化中间表示。`schemas/` 目录定义了数据、评论、索引、分析、问题缓存和检索缓存对象。最终 `ProductAnalysisResponse` 同时包含 extraction、question intents、questions、retrievals、retrieval quality、retrieval runtime、aggregates、report、trace、replay summary、artifact bundle 和 warnings。

这种设计使错误定位可以分解到具体阶段。如果最终建议不合理，可以检查评论抽样是否偏置、方面抽取是否错误、问题是否过宽、召回是否缺证据、reranker 是否误排，或报告后处理是否校准不足。与黑盒摘要相比，这种对象化链路更适合人工审查、消融实验和后续模型迭代。

5 实验设置

5.1 实验目标

本文实验定位为系统验证，而非完整 benchmark。实验目标包括四点：验证端到端工作流是否可运行；验证各阶段产物是否结构化且可审查；验证真实模型链路与降级链路是否能够通过运行策略区分；验证评估模块是否能够在有标注和无标注条件下输出可用指标。

5.2 运行环境与软件栈

项目基于 Python 3.11+，当前验证环境为 Python 3.12。核心依赖包括 FastAPI、Pydantic、Pydantic Settings、LangChain、LangGraph、OpenAI SDK、Qdrant Client、Transformers、Torch、Pillow、orjson 和 pytest。

API 入口由 FastAPI 提供，批处理入口由 `04_scripts/run_workflow.py` 提供，单商品工作流由 LangGraph 编排。LLM 调用通过 OpenAI 兼容 Qwen 网关完成；检索层支持本地 JSON 索引和 Qdrant 索引；图像 embedding 默认使用 CLIP；图像候选重排默认可使用 Qwen-VL。

5.3 数据与产物

原始商品数据位于 `01_data/01_raw_products/products/`，人工审核用中文数据可导出到 `01_data/02_audit_zh_products/products/`。系统产物按阶段写入 `02_outputs/`：标准化证据包写入 `1_normalized/`，评论方面写入 `2_aspects/`，索引与缓存写入 `3_indexes/`，分析报告写入 `4_reports/`，feedback 和 replay 写入 `5_feedback/` 与 `5_replay/`，HTML 报告写入 `6_html/`，运行 manifest 写入 `7_manifests/`。

本文不把当前数据样例描述为冻结公开 benchmark。当前数据用于验证系统链路、schema 产物和评估接口；后续若要比较不同检索策略，需要补充多品类人工标注集。

5.4 工作流协议

标准单商品协议如下：

1. 扫描数据集并生成概览。
2. 标准化目标商品证据包。
3. 为目标商品构建文本和图像索引。
4. 抽样并抽取评论方面。
5. 将方面规划为检索问题。
6. 对文本和图像 route 执行粗召回。
7. 对候选证据进行文本或多模态重排。
8. 聚合方面统计并生成报告。
9. 构建 claim attribution、trace、feedback/replay 和 manifest。
10. 可选导出 HTML 报告。

核心命令如下：

```
.venv/bin/python 04_scripts/run_workflow.py --category backpack --product-id backpack_010 --output-form
```

离线验证可关闭 LLM：

```
.venv/bin/python 04_scripts/run_workflow.py --category backpack --product-id backpack_010 --no-llm --ou
```

真实多模态运行态验证可使用：

```
.venv/bin/python 04_scripts/validate_multimodal_runtime.py --category backpack --product-id backpack_01
```

5.5 评估指标

当 manifest 或 analysis artifact 包含 `retrieval_relevance` 标注时，评估模块计算 Recall@1、Recall@3、Recall@5、MRR、NDCG@3 和 NDCG@5。这些指标是信息检索评估中的常用排序与召回指标 [13]。相关证据可以是 `region_id`、`text_block_id`、`image_id` 或 `evidence_id`。

在没有人工 gold labels 时，系统仍统计运行质量指标，包括 informative review 数、aspect 数、question 数、平均置信度、retrieval evidence 数量、evidence coverage、score drift、conflict risk、claim support rate、claim grounded rate、citation precision、citation contradiction rate，以及 text、image、mixed 的 route contribution。

5.6 自动化测试

自动化测试覆盖 API、数据集服务、评论服务、问题生成、索引后端、embedding 与 reranker、检索服务、分析服务、HTML 导出、manifest 评估、运行策略、运行进度、工作流脚本和多模态验证脚本。当前代码基线下执行：

```
.venv/bin/python -m pytest
```

结果为 166 项测试全部通过。该结果说明当前源码、提示词注册、schema、工作流脚本和评估服务在工程层面保持一致。

6 结果与分析

6.1 端到端闭环结果

当前系统已经形成从原始商品目录到结构化 VOC 报告的端到端闭环。`run_workflow.py` 能够将数据概览、标准化、索引构建和分析串联为单商品流程；`validate_multimodal_runtime.py` 能够检查运行时是否启用真实多模态链路；`export_html_report.py` 能够将分析产物导出为可审查页面；`evaluate_manifests.py` 能够读取 run manifest 并计算评估指标。

这表明系统已经不再是若干独立脚本，而是一个以服务层、schema 和 artifact 为共同边界的分析框架。即使在 `--no-llm` 离线模式下，系统仍能输出同构的方面、问题、检索和报告对象，从而支持调试和回归测试。

6.2 测试结果

在当前代码基线下，完整测试套件共收集 166 项测试，全部通过。测试覆盖范围包括数据集扫描与标准化、评论抽取、问题生成、索引后端、embedding 和 reranker、检索服务、分析服务、HTML 导出、manifest 评估、运行策略、运行进度、工作流脚本和多模态运行态验证。

Table 1: 当前代码基线下的系统验证结果。

验证对象	结果	说明
自动化测试	166/166 通过	当前实现与测试断言一致
API 层	通过	dataset、index、reviews、analysis 路由可测
检索层	通过	本地索引、Qdrant 抽象、embedding、reranker 和召回逻辑可测
生成层	通过	问题生成、报告生成、归因、HTML 和 replay 相关逻辑可测
工作流脚本	通过	run workflow、interactive workflow 和 runtime validation 可测

6.3 结构化产物分析

系统输出不是单段自然语言，而是一组可追踪对象。一次完整分析通常包含 extraction、question intents、questions、retrievals、retrieval quality、retrieval runtime、aggregates、report、trace、replay summary 和 artifact bundle。每个 retrieval 记录保留来源问题、来源方面、期望 evidence routes 和召回证据；每个召回证据保留文本块 ID、图片 ID、区域 ID、来源 section、语言、embedding score 和 rerank score。

这种结构带来两个结果。第一，报告可审查。strengths、weaknesses、controversies 和 suggestions 可以追踪到评论证据和商品证据。第二，错误可定位。若某条建议不可靠，开发者可以检查是评论抽取错误、问题规划过宽、检索候选缺失、reranker 排序错误，还是报告后处理校准不足。

6.4 问题规划层的作用

本文方法的核心主张是问题规划优于直接用评论原句检索。评论原句通常包含情绪表达、背景叙事和隐含假设，直接检索容易把商品证据空间拉宽。问题规划层将评论方面拆成多个可核验意图，例如商品文案是否直接支持、商品图片是否提供视觉证据、图文证据是否共同说明真实问题或预期偏差。

该机制使检索记录天然带有解释性。每个 retrieval 都可以回到 `source_aspect_id` 和 `source_question_id`，并且 `expected_evidence_routes` 指明该问题为什么需要文本证据、图像证据或二者结合。因此，问题规划不仅提升召回可控性，也为后续评估提供了 query-level 单元。

6.5 多模态证据路线分析

系统将文本证据和图像证据作为不同 route 处理。文本证据适合验证商品名称、类目、描述、规格或保修相关主张；图像证据适合验证结构、外观、颜色、局部细节和展示方式。图像 route 同时包含整图和默认局部区域，使系统能够在不依赖人工标注框的情况下进行初步区域级证据检索。

粗召回和重排的分离使系统能够兼顾覆盖和精度。Embedding 阶段负责扩大候选，reranker 阶段负责在候选中选择更相关证据。对于图像候选，多模态 reranker 可以直接观察原图或区域裁剪，从而减少仅靠代理文本判断图像相关性的风险。

6.6 评估接口结果

`ManifestEvaluationService` 已经支持两类评估。若运行产物包含人工检索标签，系统能够计算 Recall@K、MRR 和 NDCG；若没有人工标签，系统仍能计算结构化运行统计和 claim attribution 指标。后者包括 claim support rate、claim grounded rate、citation precision、citation contradiction rate 和 route contribution。

这一设计使当前系统具备从系统验证过渡到正式 benchmark 的接口。未来只需补充覆盖多品类、多问题和多证据类型的人工标签，即可在现有 manifest 评估框架中比较评论直召回、问题驱动召回、不同 embedding 后端和不同 reranker 后端。

6.7 当前结果边界

当前结果证明系统主链可运行、产物可审查、测试基线稳定，但仍不能等同于完整 benchmark 结论。本文尚未报告多品类冻结测试集上的统计显著性结果，也尚未系统比较不同检索策略的边际收益。因此，本节结果应被理解为系统方法验证，而不是最终性能评测。

7 讨论

7.1 为什么问题驱动检索适合商品 VOC

商品 VOC 分析的难点不只是总结评论，而是解释评论中的体验是否能被商品证据支持。问题驱动检索在评论和商品之间增加了一个中间语义层：评论方面描述用户关注点，检索问题描述需要商品证据回答的判断。这使系统既不会脱离用户真实反馈，也不会让生成模型绕过商品证据直接给出结论。

这一设计尤其适合电商场景。商品页面天然包含多模态证据，评论又天然包含主观性和噪声。将二者直接拼接给模型会牺牲可控性，而将评论方面转写为证据问题，则可以把主观反馈拆解为可检索、可评估、可归因的操作单元。

7.2 Artifact-first 的工程价值

本文系统强调 artifact-first。标准化证据包、方面抽取结果、问题缓存、检索缓存、分析报告、feedback、replay、HTML 和 manifest 都有固定结构和输出路径。这种设计的价值在于，研究者可以复查任一阶段，而不是只能阅读最终报告。

Artifact-first 也降低了论文实验成本。后续若要比较不同问题生成策略，可以复用标准化产物；若要比较不同 reranker，可以复用评论方面和问题；若要做人审，可以直接基于 HTML 和 manifest 组织标注。

7.3 与一步式大模型总结的对比

一步式大模型总结实现简单，但通常难以回答三个问题：某个结论来自哪条评论，是否被商品文本支持，是否被商品图片支持。本文方法通过结构化 schema 和 claim attribution 将这三个问题显式化。系统复杂度确实更高，但换来了可解释性、可复现性和可评估性。

对于研究型系统而言，这种取舍是必要的。商品 VOC 报告不仅要 “看起来合理”，还要能够被运营人员、产品人员或研究人员追问证据来源。若无法追踪证据，报告很难进入实际决策流程。

7.4 对后续研究的启示

当前实现说明，商品多模态分析的核心不一定是更大的生成模型，而是更好的证据组织和查询规划。后续研究可以沿四个方向扩展：第一，引入检测或分割模型，将固定 crop 区域升级为语义区域；第二，构建多品类人工标注集，系统评估 Recall@K、MRR 和 NDCG；第三，比较评论直召回、方面召回和问题驱动召回；第四，将 feedback/replay 与人工审查界面结合，形成持续修正机制。

7.5 论文写作层面的意义

本文当前版本已经可以作为系统方法论文的主体。它明确了问题定义、系统架构、模块实现、评估接口和局限性。后续若补充更大规模实验，可直接在现有章节中加入定量对照，而不需要重写方法主线。

8 结论

本文提出 Decathlon VOC Analyzer，一个面向商品图文证据与用户评论对齐的证据驱动多模态 VOC 分析系统。系统将原始商品数据标准化为可追踪证据包，将用户评论抽取为方面级 VOC 对象，再通过问题规划、多

模态召回、重排、报告生成和 claim 级归因生成结构化商品分析结果。

本文的核心结论是：商品 VOC 分析不应被简化为一步式评论摘要，而应被组织为 “评论方面建模—证据问题规划—商品图文检索—证据约束生成” 的多阶段流程。当前实现已经提供 API、批处理工作流、结构化产物、检索缓存、feedback/replay、HTML 导出、manifest 评估和自动化测试。测试结果显示当前代码基线下 166 项测试全部通过，说明系统主链和关键模块在工程层面保持一致。

未来工作将重点补充多品类人工标注 benchmark，系统比较不同检索和重排策略，引入更细粒度视觉 grounding，并将人工反馈与 replay 机制进一步闭环。

9 局限性

第一，本文当前结果仍属于系统验证，而非完整 benchmark。虽然系统已经支持 manifest 评估和自动化测试，但尚未构建覆盖多品类、多商品和多证据类型的冻结人工标注集。因此，本文不能声称系统在总体性能上优于所有基线。

第二，检索策略尚未完成系统消融。当前实现已经具备文本 route、图像 route、问题驱动检索、reranker 和缓存机制，但仍需要在统一数据集上比较评论直召回、方面直召回、问题驱动召回、不同 embedding 后端和不同 reranker 后端。

第三，视觉证据粒度仍有限。当前系统支持整图和固定比例区域 crop，但这些区域不是由语义检测、分割或 grounding 模型产生。对于鞋底结构、缝线、镜片光学细节等强局部属性，固定区域可能不足以提供精确解释。

第四，LLM 输出仍依赖提示词与 schema 约束。尽管系统提供结构化输出、后处理校准和 heuristic 兜底，但在复杂评论或跨语言场景下，方面抽取和报告生成仍可能出现误解、遗漏或过度概括。

第五，反馈闭环仍处于工程接口阶段。feedback/replay sidecar 已经存在，但尚未完成大规模人工审查流程，也未量化 replay 对后续报告质量的提升。

第六，参考文献和实验报告仍需面向正式投稿进一步扩展。本文当前更强调系统方法和实现事实，后续投稿版本应补充更系统的相关工作比较、正式图表、人工标注协议和统计显著性分析。

10 致谢

本稿基于当前仓库中的设计文档、源码实现、脚本、测试与输出产物整理而成。后续正式版本可在此补充项目指导、数据来源支持和实验环境协助等致谢信息。

11 附录

11.1 A. 关键目录与职责

目录	职责
`01_data/`	原始商品数据与中文审核数据
`02_outputs/`	标准化结果、方面结果、索引、报告、feedback、replay、HTML 与 manifest

目录	职责
03_configs/	评论抽样 profile 与运行策略配置
04_scripts/	工作流、评估、导出、验证和清理脚本
05_src/	核心源码实现
06_tests/	API、服务、脚本和工作流测试
0_docs/03_ 论文子模块文档/	基于当前源码整理的论文模块文档
论文 _md 形式/	论文分章稿、参考文献和导出脚本

11.2 B. API 入口

系统当前包含以下主要 API:

- /health
- /api/v1/dataset/overview
- /api/v1/dataset/normalize
- /api/v1/index/overview
- /api/v1/index/build
- /api/v1/reviews/extract
- /api/v1/analysis/product

这些接口覆盖数据集概览、商品标准化、索引构建、评论结构化抽取和单商品综合分析。

11.3 C. 复现命令

端到端运行:

```
.venv/bin/python 04_scripts/run_workflow.py --category backpack --product-id backpack_010 --output-form
```

离线运行:

```
.venv/bin/python 04_scripts/run_workflow.py --category backpack --product-id backpack_010 --no-llm --ou
```

验证多模态运行态:

```
.venv/bin/python 04_scripts/validate_multimodal_runtime.py --category backpack --product-id backpack_01
```

评估 manifest:

```
.venv/bin/python 04_scripts/evaluate_manifests.py 02_outputs/7_manifests
```

运行测试:

```
.venv/bin/python -m pytest
```

合并论文:

```
.venv/bin/python 论文_md形式/脚本/build_complete_paper.py
```

导出 PDF:

```
.venv/bin/python 论文_md形式/脚本/pipeline.py
```

11.4 D. 主要结构化产物

产物	默认位置	用途
标准化商品证据包	`02_outputs/1_normalized/`	后续索引和分析输入
评论方面结果	`02_outputs/2_aspects/`	问题规划和聚合输入
索引与检索缓存	`02_outputs/3_indexes/`	召回和重排复用
分析报告	`02_outputs/4_reports/`	最终结构化 VOC 结果
feedback sidecar	`02_outputs/5_feedback/`	人工反馈入口
replay sidecar	`02_outputs/5_replay/`	后续分析回放
HTML 报告	`02_outputs/6_html/`	人工审查界面
run manifest	`02_outputs/7_manifests/`	评估与复现实验入口

11.5 E. 当前论文稿适用边界

本稿适用于系统设计说明、方法章节草稿、项目答辩和后续实验规划。不适用于直接作为完整 benchmark 论文终稿，因为仍缺少多品类冻结标注集、正式消融实验和大规模人工评测。

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K"uttler, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL https://arxiv.org/abs/2005.11401.

[2] Luyu Gao, Zhuyun Dai, Panupong Pasupat, Anthony Chen, Arun Tejasvi Chaganty, Yicheng Fan, Vincent Y. Zhao, Ni Lao, Hongrae Lee, Da-Cheng Juan, and Kelvin Guu. RARR: Researching and revising what language models say, using language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16477–16508, 2023. doi: 10.18653/v1/2023.acl-long.910. URL https://doi.org/10.18653/v1/2023.acl-long.910.

[3] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever.

Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, pages 8748–8763. PMLR, 2021. URL https://proceedings.mlr.press/v139/radford21a.html.

[4] NVIDIA Developer Blog. An easy introduction to multimodal retrieval-augmented generation, 2024. URL https://developer.nvidia.com/blog/an-easy-introduction-to-multimodal-retrieval-augmented-generation/.

[5] NVIDIA Developer Blog. Build ai-ready knowledge systems using essential multimodal rag capabilities, 2024. URL https://developer.nvidia.com/blog/build-ai-ready-knowledge-systems-using-5-essential-multimodal-rag-capabilities/.

[6] Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Celine Hudelot, and Pierre Colombo. ColPali: Efficient document retrieval with vision language models, 2024. URL https://arxiv.org/abs/2407.01449.

[7] Maria Pontiki, Dimitris Galanis, John Pavlopoulos, Harris Papageorgiou, Ion Androutsopoulos, and Suresh Manandhar. SemEval-2014 task 4: Aspect based sentiment analysis. In *Proceedings of the 8th International Workshop on Semantic Evaluation*, pages 27–35, 2014. doi: 10.3115/v1/S14-2004. URL https://aclanthology.org/S14-2004/.

[8] Qdrant. Pdf retrieval at scale: Search engineering with multi-stage retrieval, 2024. URL https://qdrant.tech/documentation/tutorials-search-engineering/pdf-retrieval-at-scale/.

[9] Qdrant. Multi-vector search and multi-stage retrieval course, 2024. URL https://qdrant.tech/blog/multi-vector-search-course/.

[10] Yu Liu, Duantengchuan Li, Kaili Wang, Zhuoran Xiong, Fobo Shi, Jian Wang, Bing Li, and Bo Hang. Are LLMs good at structured outputs? a benchmark for evaluating structured output capabilities in LLMs. *Information Processing & Management*, 61(5):103809, 2024. doi: 10.1016/j.ipm.2024.103809. URL https://doi.org/10.1016/j.ipm.2024.103809.

[11] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, 2023. doi: 10.18653/v1/2023.emnlp-main.674. URL https://doi.org/10.18653/v1/2023.emnlp-main.674.

[12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

[13] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Sch"utze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. URL https://nlp.stanford.edu/IR-book/.